Database Schema

1. Overview

ArtFrame is a full-stack e-commerce platform for selling framed artwork. The database is built on MongoDB using Mongoose ODM. It consists of six collections: **User, Product, Category, Cart, Order, and Review**. All collections use MongoDB's default ObjectId (`_id`) as the primary key, and most are linked by ObjectId references.

---

2. Collections

2.1 User

Stores registered user accounts. The `role` field controls access level (user vs. admin).

Field	Type	Required	Unique	Default	Description
`_id`	ObjectId	Auto	Yes	—	Primary key (auto-generated)
`name`	String	Yes	No	—	Full display name of the user
`email`	String	Yes	Yes	—	Unique email used for login
`password`	String	Yes	No	—	Bcrypt-hashed password
`role`	String	No	No	`"user"`	Either `"user"` or `"admin"`

2.2 Category

Represents a product collection/genre (e.g., Cars, Sports, Music). Products belong to one category.

Field	Type	Required	Unique	Default	Description
`_id`	ObjectId	Auto	Yes	—	Primary key (auto-generated)
`name`	String	Yes	Yes	—	Unique category name (e.g., Cars)
`description`	String	Yes	No	—	Short description shown in the Collections page

---

2.3 Product

Core catalog item. Each product belongs to a category. Images are stored as base64 strings or URLs.

Field	Type	Required	Unique	Default	Constraints	Description
`_id`	ObjectId	Auto	Yes	—	—	Primary key
`name`	String	Yes	No	—	trim	Product display name

Field	Type	Required	Unique	Default	Constraints	Description
`description`	String	No	No	`""`	—	Long-form product description
`price`	Number	Yes	No	—	min: 0	Price in EGP
`image`	String	No	No	`""`	—	Base64 or URL of product image
`category`	ObjectId → Category	No	No	null	ref	FK to Category collection
`stock`	Number	No	No	0	min: 0	Available inventory count
`material`	String	No	No	`""`	—	Frame material (e.g., Wood)
`createdAt`	Date	Auto	No	—	timestamp	Auto-set by Mongoose
`updatedAt`	Date	Auto	No	—	timestamp	Auto-updated by Mongoose

2.4 Cart

A single persistent cart per user. Items are an embedded array of sub-documents. The cart is cleared after a successful checkout.

Parent document:

Field	Type	Required	Unique	Default	Description
`_id`	ObjectId	Auto	Yes	—	Primary key
`user`	ObjectId → User	Yes	Yes	—	FK to User; enforced unique (one cart per user)
`items`	[CartItem]	No	No	[]	Embedded array of cart line items
`createdAt`	Date	Auto	No	—	Mongoose timestamp
`updatedAt`	Date	Auto	No	—	Mongoose timestamp

Embedded CartItem sub-document:

Field	Type	Required	Constraints	Default	Description
`_id`	ObjectId	Auto	—	—	Sub-document ID
`product`	ObjectId → Product	Yes	ref	—	FK to Product
`quantity`	Number	No	min: 1	1	Number of units added

2.5 Order

A placed order. Items array is a snapshot of the product and price at time of purchase. The status field follows the lifecycle: `pending → processing → shipped → delivered` (or `cancelled`).

Parent document:

Field	Type	Required	Constraints	Default	Description
`_id`	ObjectId	Auto	—	—	Primary key
`user`	ObjectId → User	Yes	ref	—	FK to User who placed the order
`items`	[OrderItem]	Yes	—	[]	Array of ordered line items
`totalPrice`	Number	Yes	—	—	Grand total in EGP
`status`	String (enum)	No	enum (5)	`"pending"`	Order lifecycle status
`shippingAddress`	Object	No	—	—	Embedded shipping address
`createdAt`	Date	Auto	—	—	Mongoose timestamp
`updatedAt`	Date	Auto	—	—	Mongoose timestamp

Embedded OrderItem sub-document:

Field	Type	Required	Constraints	Default	Description
`_id`	ObjectId	Auto	—	—	Sub-document ID
`product`	ObjectId → Product	Yes	ref	—	FK reference to Product
`quantity`	Number	Yes	min: 1	—	Quantity ordered
`price`	Number	Yes	—	—	Price per unit at time of order

Embedded ShippingAddress object:

Field	Type	Required	Description
`street`	String	No	Street address (includes first + last name from checkout form)
`city`	String	No	City
`country`	String	No	Country (defaults to Egypt)

Order status enum values:

Status	Description
`pending`	Order placed, awaiting admin action. Default status.
`processing`	Admin has acknowledged and is preparing the order.
`shipped`	Order dispatched to courier.
`delivered`	Order received by customer. Terminal state — cannot be changed.
`cancelled`	Order cancelled. Terminal state — cannot be changed.

2.6 Review

Product reviews written by logged-in users. Each review targets a specific product.

Field	Type	Required	Constraints	Default	Description
`_id`	ObjectId	Auto	—	—	Primary key
`user`	ObjectId → User	Yes	ref	—	User who wrote the review
`product`	ObjectId → Product	Yes	ref	—	Product being reviewed
`rating`	Number	Yes	min: 1, max: 5	—	Star rating (1–5)
`comment`	String	No	trim	`""`	Written review body
`createdAt`	Date	Auto	—	—	Mongoose timestamp
`updatedAt`	Date	Auto	—	—	Mongoose timestamp

3. Collection Relationships

All relationships are implemented via ObjectId references (Mongoose populate). MongoDB itself does not enforce foreign key constraints — referential integrity is managed at the application layer.

Collection	Field	References	Relationship Type	Description
Cart	`user`	User	One-to-One	Each cart belongs to exactly one user
Cart	`items[].product`	Product	Many-to-Many	Cart items reference products
Order	`user`	User	Many-to-One	Many orders can belong to one user
Order	`items[].product`	Product	Many-to-Many	Order items snapshot product info
Product	`category`	Category	Many-to-One	Many products belong to one category
Review	`user`	User	Many-to-One	One user can leave many reviews
Review	`product`	Product	Many-to-One	One product can have many reviews

4. Constraints & Validation Rules

Collection	Field	Rule
User	`email`	Must be unique across the collection
User	`role`	String; defaults to `"user"`; should be `"user"` or `"admin"`
Category	`name`	Required; must be unique
Category	`description`	Required
Product	`price`	Required; min value 0

Collection	Field	Rule
Product	`stock`	Default 0; min value 0. UI disables Add-to-Cart when stock = 0
Cart	`user`	Required; unique — enforces one cart per user
Cart	`items.quantity`	Minimum 1
Order	`status`	Enum: `["pending", "processing", "shipped", "delivered", "cancelled"]`
Order	`items.quantity`	Required; min 1
Order	`items.price`	Required; price snapshot at time of order
Review	`rating`	Required; integer 1–5
Review	`comment`	Optional but required by UI validation before submission

5. Indexes

MongoDB automatically creates a unique index on `_id` for every collection. The following additional indexes are defined via Mongoose schema options.

Collection	Field(s)	Index Type	Purpose
All	`_id`	Unique (default)	Primary key lookup
User	`email`	Unique	Enforce one account per email; login lookup
Category	`name`	Unique	Enforce no duplicate category names
Cart	`user`	Unique	Enforce one cart per user; fast cart fetch

6. Key Data Flows

6.1 Add to Cart

1. Client sends `POST /api/cart` with `productId` and `quantity`.
2. Server finds or creates the Cart document for the authenticated user.
3. If the product already exists in `items[]`, its quantity is incremented.
4. Otherwise, a new CartItem sub-document is pushed.
5. The updated cart (with populated product details) is returned.

6.2 Checkout

1. Client sends `POST /api/orders/checkout` with a `shippingAddress`.
2. Server reads the user's Cart and populates product prices.
3. A new Order document is created with a snapshot of items and prices.
4. The Cart is cleared (all items removed).
5. Each product's stock is decremented by the ordered quantity.
6. Order status starts as `"pending"`.

6.3 Order Status Update (Admin)

1. Admin sends `PUT /api/orders/:id/status` with the new status.
2. Server validates that the transition is allowed (`delivered` and `cancelled` are terminal states — the UI disables the dropdown).
3. The Order document's `status` field is updated.